

Introduction to Design Patterns

Structural patterns — Advanced Programming

Ayoub FRIHAOUI

`ayoub.frihaoui@lecnam.net`

Conservatoire national des arts et métiers (CNAM Paris)

Original material © Rezak AZIZ, CNAM Paris.

Source: <https://rezakaziz.github.io/PROG-AVANC/>

What is a Design Pattern?

Definition

A **Design Pattern** is a **classical solution** to a **recurring** software design problem.

Fundamental principle

A Pattern acts as a *conceptual plan*. It describes:

- the elements at play;
- their relations;
- and how they collaborate to solve a given problem.

Use in design

- Patterns are not ready-made libraries.
- They guide **architectural thinking** before implementation.
- One Pattern can lead to several concrete implementations.

To remember

Patterns are not code to copy, but **reusable, adaptable design schemes**.

History of Design Patterns

Origin of the concept

The term Design Pattern does not denote a sudden invention, but the formalisation of solutions already used for a long time.

First appearance

The idea of “Pattern” comes from the architect **Christopher Alexander**, in his book *A Pattern Language: Towns, Buildings, Construction* (1977). There he describes a language of urban design built on recurring units: the Patterns.

Transposition to computing

In 1994, four authors: **Erich Gamma, John Vlissides, Ralph Johnson, and Richard Helm**, publish the founding book: *Design Patterns: Elements of Reusable Object-Oriented Software*.

This book, better known as “Gang of Four” (GoF), presents **23 Patterns** for object-oriented design.

Since then, many other Patterns have been discovered.

Why learn Design Patterns?

An implicit practice

- You can develop for years without having studied Design Patterns.
- Most developers already use them **without knowing it**.

Why study them then?

- Understand and **name** the schemes you already apply;
- To improve your ability to **design, refactor and discuss** code.

A conceptual toolbox

A true **toolbox of reliable solutions** to classical OO design problems. They teach

- separation of responsibilities,
- weak coupling,
- reuse and extensibility.

A common language

Patterns offer a shared vocabulary between developers. Saying: “Let’s use a *Singleton*” is enough to understand each other, without rewriting the whole code structure.

Patterns, algorithms and the structure of a Pattern

Pattern $\neq$ Algorithm

- An **algorithm** describes a precise sequence of steps leading to a result.
- A **Pattern** describes an organisation of classes and objects, adaptable as needed.

Typical structure of a Pattern

Each Pattern is generally described in the following sections:

- **Intent** — Goal or Role.
- **Problem** — Context and Motivations.
- **Solution**
- **Structure** — classes, roles, relations;
- **Code example**
- **Applicability** — When?
- **Advantages and Drawbacks**
- **Practical links** — variants and relations with other Patterns.

Example of a Design Pattern: Singleton

Intent

The **Singleton** is a creational Design Pattern that guarantees a class has only one instance in the system, while providing a **global access point** to that instance.

Goal

- Avoid duplicating a unique object (e.g. database connection, logger).
- Offer centralised, controlled access to that instance.

Problem addressed by the Singleton

Problems

The Singleton solves two major problems:

- ➊ **Guarantee a single instance** for classes managing shared resources.
- ➋ **Offer secure global access** to that instance.

Difficulty

A classical constructor always creates a new object.

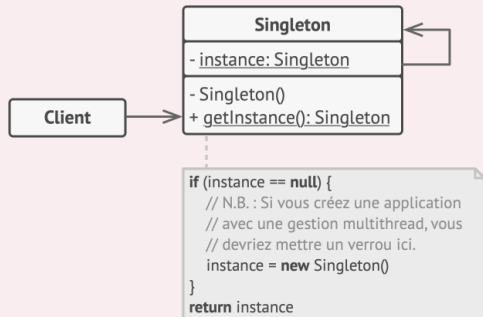
Proposed solution

Singleton principle

- ① Make the constructor **private** to block external instantiation.
- ② Provide a **static** method `getInstance()` that creates and returns the unique instance.
- ③ Keep that instance in a **static** attribute.

Structure of the Singleton

Simplified diagram



The client code cannot create an object directly: it accesses the unique instance via `getInstance()`.

```
class Database {  
    private static Database instance;  
  
    private Database() {}  
    public static Database getInstance() {  
        if (instance == null) {  
            synchronized(Database.class) {  
                if (instance == null) {  
                    instance = new Database  
                        ();  
                }  
            }  
        }  
        return instance;  
    }  
}
```

- When a class must have a single instance (e.g. database, logger, configuration manager).
- When controlled global access is needed.
- To replace a global variable with a safe mechanism.

Advantages and drawbacks

Advantages

- Guarantees instance uniqueness.
- Offers a global access point.
- Allows lazy instantiation.

Drawbacks

- Does not respect the single responsibility principle (two problems for the database: global access uniqueness and business logic (connection, query)).
- Makes unit tests harder.
- Must be protected against multithreading.
- May hide an over-coupled design.

Watch out: criticisms of Design Patterns

1. Workarounds for bad languages

Some Patterns compensate for the limits of a language.

- In modern languages, several patterns become superfluous.

2. Inefficient solutions

Applying a Pattern “by the book” without adapting to the context leads to rigid, needlessly complex architectures.

3. Unjustified use

“If all you have is a hammer, everything looks like a nail.” Beginners often want to apply Patterns everywhere, even when simple code is enough.

To remember

Patterns are **tools**, not obligations. They must be understood, adapted and used with discernment.

The big families of Design Patterns

General classification

Design Patterns are classified by their **main goal** in the structure of software. Three big families are traditionally distinguished.

1. Creational patterns

Provide object-creation mechanisms that increase code flexibility and reuse. Examples: Singleton, Factory, Builder, Prototype

2. Structural patterns

How to assemble objects and classes into bigger structures, while keeping them flexible and efficient. Examples: Decorator, Composite, Facade, Adapter, Bridge, Proxy, ...

3. Behavioural patterns

Set up efficient communication and distribute responsibilities between objects. Examples: Visitor, Template Method, Observer, Strategy, ...

Structural patterns

Structural Patterns

Definition

Structural Patterns describe how to **compose classes and objects** to form bigger, more flexible structures.

Main structural Patterns

- **Adapter** — lets objects with incompatible interfaces collaborate.
- **Bridge** — splits a big class into two hierarchies.
- **Composite** — arranges objects into trees.
- **Decorator** — dynamically adds new behaviours.
- **Facade** — provides an interface with simplified access to a complex subsystem.
- **Flyweight** — shares objects to save memory.
- **Proxy** — uses a substitute for an object. (defers creation)

Section 1

Composite

Problem to solve

Concrete example

We want to compute the total cost of an order containing:

- simple products,
- boxes containing other products,
- boxes nested inside each other.

Difficulty

- You may think of unpacking all boxes, then taking all objects and summing.
- But,
 - ▶ The structure is **hierarchical and variable**.
 - ▶ Impossible to write a simple loop: you must know the nature of each object and its nesting level.

Intent

The **Composite** Pattern arranges objects in a **tree structure**, to treat in the same way:

- simple objects (leaves),
- and composed objects (containers).

Solution of the Composite Pattern

Principle

Define a **common interface** for simple and composed objects:

- **Leaf**: returns its own result (e.g. price).
- **Container (Composite)**: delegates to children, sums or combines their results.

Recursive approach

Each composite walks its children, calls their common method, and aggregates the results.

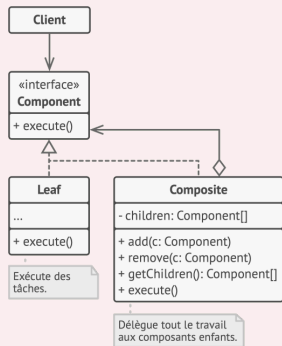
Key advantage

The client no longer distinguishes:

- a leaf from a composite;
- a simple element from a complex set.

Everything is handled through one interface.

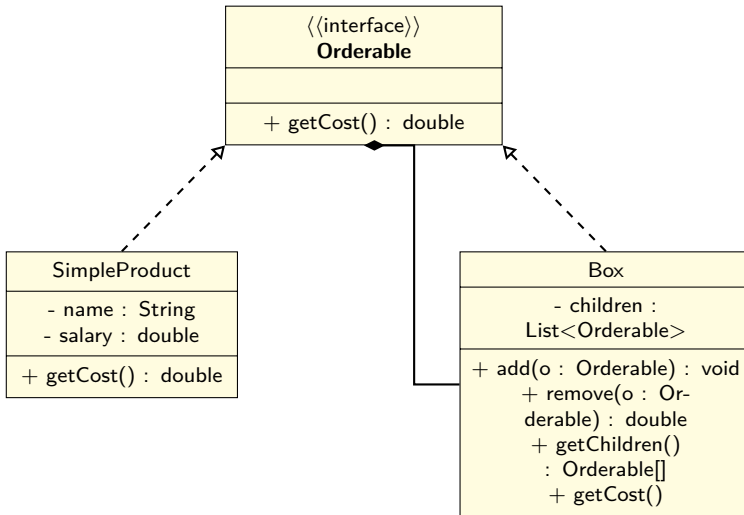
Structure of the Composite



Principle

- The Component interface describes the operations common to the simple and complex objects of the tree.
- A Leaf is a basic element of a branch with no sub-element.
- The Composite is an element composed of sub-elements.
- Each composite delegates calls to its children and aggregates their results.

Example of Composite



Advantages and limits of the Composite

Advantages

- Simplifies managing hierarchical structures;
- Exploits **polymorphism**;
- Opens the way to the OCP principle.

Drawbacks

- Hard to define a common interface when elements are very different;
- Risk of an over-generic, unreadable interface.

When to use it?

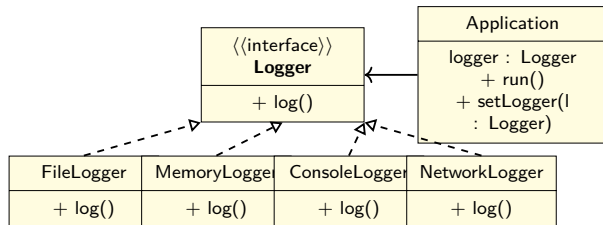
The *Composite* is ideal for modelling nested/tree objects and uniformising their handling:

- files and folders,
- boxes containing other boxes,
- nested graphical elements, etc.

Section 2

Decorator

Problem to solve



Suppose we sometimes want to encrypt logs, add a timestamp, and record them both on the console and in a file, without modifying the **Application** class.

What solution do you propose?

General intent

Intent

The **Decorator** Pattern (also called Wrapper) dynamically **adds new behaviours** to an existing object, without modifying its source code.

Key idea

An object is “wrapped” in another object implementing the same interface, which can enrich, filter or transform calls before or after delegation.

Solution of the Decorator Pattern

Composition principle

Rather than extending a class:

- Create a **decorator** referencing another object;
- It implements the same interface and delegates calls to it;
- It can run code before or after that delegation.

Difference with inheritance

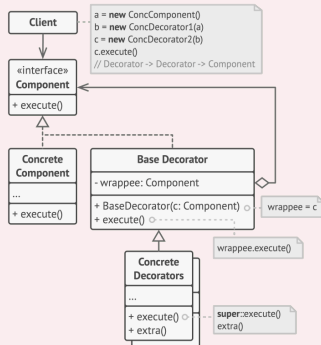
- Inheritance is static: behaviour is fixed at compilation;
- Composition is dynamic: behaviour changes at execution.

→ The Decorator relies on *composition*, not inheritance.

Key advantage

An object's behaviour can be modified or enriched **dynamically**, at execution time.

Structure of the Decorator

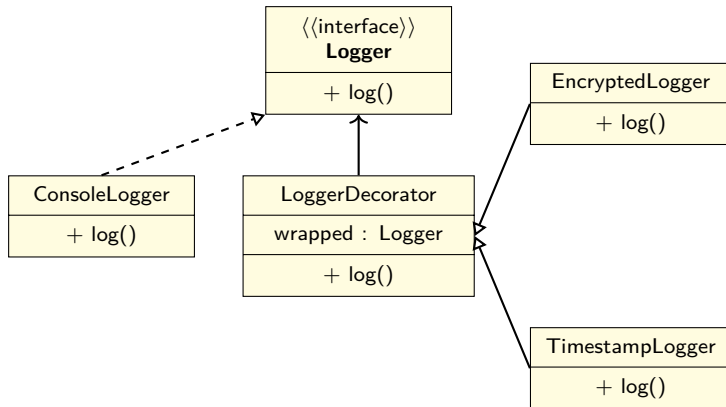


Principle

The client handles the decorated object through the common interface, without knowing how many decorators are stacked.

- **Component** defines the interface common to all;
- **Concrete component** defines the base behaviour;
- **Base Decorator** references a component (already decorated or not);
- **Concrete Decorators** add specific behaviours to execute.

Example of Decorator



Advantages and limits of the Decorator

Advantages

- Dynamic behaviour addition without inheriting;
- Flexible combination of features;
- Respects the **single responsibility** principle.

Drawbacks

- Layer configuration sometimes complex;
- Hard to remove a specific decorator;
- Multiple stackings hard to read.

When to use it?

- To extend an object's behaviour at execution time;
- When inheritance becomes too rigid or forbidden;
- When several extensions must be freely combined.

Using Patterns

Patterns as a common language

Key idea

Patterns offer a **shared vocabulary** between designers/developers:

- “It is a Decorator”
- “It is a Facade”
- “It is a Composite”

Advantage

Fluid communication between developers: we understand each other without reading all the code.

Good practices for using patterns

Do

- Use patterns to **clarify the design**;
- Refactor towards a pattern when a recurring structure emerges;
- Document pattern choices in code or design.

Avoid

- “Patternitis”: wanting to fit a pattern everywhere;
- Useless over-design;
- Using a pattern just to “look professional”.

Summary — Using Patterns

Essential to remember

- Patterns are not ready-made recipes, but **thinking models**.
- They ease code communication and maintenance.
- They must be used with discernment and understanding.

In practice

In your designs: identify recurring structures, give them a name (the pattern), and share that vocabulary with your team.